



PlaceOrder MicroService

- Application Name : **MyPlaceOrderMS**
- Ports : 7000 , 7100 , 7200

Steps to Develop MyPlaceOrderMS:

A)Setup the Database:

1) Create and open the database

```
drop database jlcordersdb;  
create database jlcordersdb;  
use jlcordersdb;
```

2) Create the tables required

```
drop table myorders;
```

```
create table myorders(  
order_id int primary key,  
order_date char(25),  
user_id char(10),  
total_qty int,  
total_cost double,  
status char(15)  
);
```

```
drop table myorderitems;
```

```
create table myorderitems(  
order_Item_id int primary key,  
order_id int,  
book_id int,  
qty int,  
cost double  
);
```

```
drop table mybookinventory;
```

```
create table mybookinventory(  
book_id int primary key,  
books_available int  
);
```



3) Insert some sample records

```
insert into myorders values(5001,'20-Oct-21','U-123',5,5000,"New");
```

```
insert into myorderitems values(1,5001,101,2,2000);
```

```
insert into myorderitems values(2,5001,102,3,3000);
```

```
insert into mybookinventory values(101,50);
```

```
insert into mybookinventory values(102,50);
```

```
insert into mybookinventory values(103,50);
```

```
insert into mybookinventory values(104,50);
```

```
insert into mybookinventory values(105,50);
```

```
insert into mybookinventory values(106,50);
```

```
insert into mybookinventory values(107,50);
```

```
insert into mybookinventory values(108,50);
```

```
insert into mybookinventory values(109,50);
```

4) Select and see the sample records

```
select * from myorders;
```

```
select * from myorderitems;
```

```
select * from mybookinventory;
```

B)Setup the Boot Project

5) Copy BookPriceMS and Paste as PlaceOrderMS

6) Remove All except the following

- a) pom.xml
- b) application.properties
- c) JLCBookPriceConfig.java
- d) BookPriceApplication.java

7) Update pom.xml

*** Change Names

```
<groupId>com.jlcindia.placeorder</groupId>
```

```
<artifactId>PlaceOrderMS</artifactId>
```

```
<version>1.0</version>
```

```
<packaging>jar</packaging>
```

```
<name>MyPlaceOrderMS</name>
```

```
<url>https://www.jlcindia.com</url>
```

```
<finalName>MyPlaceOrderMS</finalName>
```



8) Update application.properties

```
spring.application.name=MyPlaceOrderMS
server.port=7000
spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb
```

9) Rename the Package to com.jlcindia.placeorder

10) Rename Spring Bean Configuration Class

```
*** JLCPlaceOrderConfig.java
```

11) 11)Rename BootApplication Starting Class.

```
*** PlaceOrderApplication.java
```

C) Develop the PlaceOrder MicroService Code:

12) Write the JPA Entity

```
*** Order.java
*** OrderItem.java
*** BookInventory.java
```

13) Write DAO Layer components

```
*** OrderDAO.java
*** OrderItemDAO.java
*** BookInventoryDAO.java
```

14) Write DTO

```
*** OrderInfo.java
```

15) Write Service Layer components

```
*** OrderService.java
*** OrderServiceImpl.java
```

16) Write Controller Layer component

```
*** OrderController.java
```



```
@PostMapping("/placeOrder")
public void placeOrder(@RequestBody OrderInfo orderInfo)

@GetMapping("/myorders/{userId}")
public List<Order> getOrdersByUserId(@PathVariable String userId)
```

17) Start the Boot Application

18) Open the Swagger UI :

<http://localhost:7000/swagger-ui.html>

19) Test

Post : <http://localhost:7000/placeOrder>
GET : <http://localhost:7000/myorders/U-123>

PlaceOrderMS: Files required

1. OrderController.java	2. OrderInfo.java
3. OrderService.java	4. OrderServiceImpl.java
5. OrderDAO.java	6. OrderItemDAO.java
7. BookInventoryDAO.java	8. Order.java
9. OrderItem.java	10. BookInventory.java
11. JLCPlaceOrderConfig.java	12. PlaceOrderApplication.java
13. application.properties	14. pom.xml

1.OrderController.java

```
package com.jlcindia.placeorder;

import java.util.List;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
```



```
import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@CrossOrigin // CORS
@RestController
public class OrderController {
    static Logger log = LoggerFactory.getLogger(OrderController.class);

    @Autowired
    OrderService orderService;

    @PostMapping("/placeOrder")
    @ApiOperation(value = " placeOrder", response = void.class, notes = "placeOrder")
    public void placeOrder(@RequestBody OrderInfo orderInfo) {
        log.info("---OrderController---placeOrder()-----");

        orderService.placeOrder(orderInfo);
    }

    @GetMapping("/myorders/{userId}")
    @ApiOperation(value = " getOrdersByUserId", response = List.class, notes = "Return Orders belongs User")
    public List<Order> getOrdersByUserId(@PathVariable String userId){
        log.info("---OrderController---getOrdersByUserId()-----");

        List<Order> myorders= orderService.getOrdersByUserId(userId);
        return myorders;
    }
}
```

2.OrderInfo.java

```
package com.jlcindia.placeorder;

import java.util.List;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



```
public class OrderInfo {
    private Order order;
    private List<OrderItem> itemsList;

    public OrderInfo() {
    }

    public OrderInfo(Order order, List<OrderItem> itemsList) {
        super();
        this.order = order;
        this.itemsList = itemsList;
    }

    //Setters and Getters
}
```

3. OrderService.java

```
package com.jlcindia.placeorder;

import java.util.List;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public interface OrderService {
    public void placeOrder(OrderInfo orderInfo) ;
    public List<Order> getOrdersByUserId(String userId);
}
```

4. OrderServiceImpl.java

```
package com.jlcindia.placeorder;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



@Service

@Transactional

```
public class OrderServiceImpl implements OrderService {  
    static Logger log = LoggerFactory.getLogger(OrderServiceImpl.class);
```

@Autowired

OrderDAO orderDAO;

@Autowired

OrderItemDAO orderItemDAO;

@Autowired

BookInventoryDAO bookInventoryDAO;

@Override

```
public void placeOrder(OrderInfo orderInfo) {  
    log.info("---OrderServiceImpl---placeOrder()-----");
```

//1. Save the Order

```
    Order myorder=orderInfo.getOrder();  
    myorder=orderDAO.save(myorder);//5002  
    int orderId=myorder.getOrderId();
```

//2.Save OrderItems

```
    for(OrderItem myitem:orderInfo.getItemsList()) {  
        myitem.setOrderId(orderId);  
        orderItemDAO.save(myitem);  
    }
```

//3.Update Inventory at Two Places

```
    RestTemplate bookSearchRest=new RestTemplate();  
    String endpoint="http://localhost:8000/updateBookInventory";
```

```
    for(OrderItem myitem:orderInfo.getItemsList()) {  
        Integer bookId = myitem.getBookId();  
        BookInventory bookInventory=bookInventoryDAO.findById(bookId).get();  
        bookInventory.setBooksAvailable(bookInventory.getBooksAvailable()-1);
```

//3A. Update Inventory of PlaceOrderMS Locally

```
        bookInventoryDAO.save(bookInventory);
```

//3B. Update Inventory of BookSearchMS Remotely

```
        bookSearchRest.put(endpoint, bookInventory);  
    }
```



```
}

@Override
public List<Order> getOrdersByUserId(String userId) {
    log.info("---OrderServiceImpl---getOrdersByUserId()-----");

    List<Order> orderList= orderDAO.findOrdersByUserId(userId);
    return orderList;
}

}
```

5. OrderDAO.java

```
package com.jlcindia.placeorder;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Repository
public interface OrderDAO extends JpaRepository<Order, Integer> {
    List<Order> findOrdersByUserId(String userId);
}
```

6. OrderItemDAO.java

```
package com.jlcindia.placeorder;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Repository
public interface OrderItemDAO extends JpaRepository<OrderItem, Integer> {
    List<OrderItem> findOrderItemsByOrderId(Integer orderId);
}
```



7. BookInventoryDAO.java

```
package com.jlcindia.placeorder;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Repository
public interface BookInventoryDAO extends JpaRepository<BookInventory, Integer> {

}
```

8. Order.java

```
package com.jlcindia.placeorder;

import javax.persistence.*;
import io.swagger.annotations.ApiModel;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Entity
@Table(name = "myorders", schema = "jlcordersdb")
@ApiModel("Order contains all the Order Details")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "orderId_generator")
    @SequenceGenerator(name = "orderId_generator", sequenceName = "myorderId_gen",
        initialValue = 5002, allocationSize = 1)
    @Column(name = "order_id")
    private Integer orderId;

    @Column(name = "order_date")
    private String orderDate;

    @Column(name = "user_id")
    private String userId;

    @Column(name = "total_qty")
    private int totalQty;
}
```



```
@Column(name = "total_cost")  
private double totalCost;
```

```
@Column(name = "status")  
private String status;
```

```
//Constructors  
//Setters and Getters  
}
```

9. OrderItem.java

```
package com.jlcindia.placeorder;
```

```
import javax.persistence.*;  
import io.swagger.annotations.ApiModel;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
* */
```

```
@Entity
```

```
@Table(name = "myorderitems", schema = "jlcordersdb")
```

```
@ApiModel("OrderItem contains OrderItem Details")
```

```
public class OrderItem {
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator =  
"orderItemId_generator")
```

```
@SequenceGenerator(name = "orderItemId_generator", sequenceName =  
"myorderId_gen", initialValue = 3, allocationSize = 1)
```

```
@Column(name = "order_item_id")
```

```
private Integer orderItemId;
```

```
@Column(name = "order_id")
```

```
private Integer orderId;
```

```
@Column(name = "book_id")
```

```
private Integer bookId;
```

```
@Column(name = "qty")
```

```
private int qty;
```

```
@Column(name = "cost")
```

```
private double cost;
```



```
//Constructors  
//Setters and Getters  
}
```

10. BookInventory.java

```
package com.jlcindia.placeorder;  
  
import javax.persistence.*;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
@Entity  
@Table(name="mybookinventory",schema = "jlcordersdb")  
@ApiModel("BookInventory contains Inevntory Details")  
public class BookInventory {  
  
    @Id  
    @Column(name="book_id")  
    private Integer bookId;  
  
    @Column(name="books_available")  
    private int booksAvailable;  
  
    //Constructors  
    //Setters and Getters  
}
```

11.JLCPlaceOrderConfig.java

```
package com.jlcindia.placeorder;  
  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.context.annotation.Bean;  
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;  
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;  
  
import com.google.common.base.Predicates;  
  
import springfox.documentation.builders.PathSelectors;  
import springfox.documentation.builders.RequestHandlerSelectors;  
import springfox.documentation.service.ApiInfo;  
import springfox.documentation.service.Contact;  
import springfox.documentation.spi.DocumentationType;
```



```
import springfox.documentation.spring.web.plugins.Docket;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
public class JLCPlaceOrderConfig implements WebMvcConfigurer {

    private ApiInfo getApiDetails() {
        return new ApiInfo("JLC BookStore- API",
            "PlaceOrderMS- API - part of BookStore", "1.0",
            "Free to use ",
            new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"),
            "API Under Free Licence",
            "https://www.jlcindia.com");
    }

    @Bean
    public Docket api() {
        return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())
            .apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))
            .build()
            .apiInfo(getApiDetails());
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        //enabling swagger-ui
        registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-INF/resources/");

        registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-INF/resources/webjars/");
    }
}
```



12. PlaceOrderApplication.java

```
package com.jlcindia.placeorder;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import springfox.documentation.swagger2.annotations.EnableSwagger2;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableSwagger2
public class PlaceOrderApplication {
    static Logger log=LoggerFactory.getLogger(PlaceOrderApplication.class);

    public static void main(String args[]) {
        log.info(" PlaceOrderMS - Begin ");
        SpringApplication.run(PlaceOrderApplication.class, args);
        log.info(" PlaceOrderMS - End ");
    }
}
```

13. application.properties

```
spring.application.name=MyPlaceOrderMS

server.port=7000

spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```



14. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
  <groupId>com.jlcindia.placeorder</groupId>
<artifactId>PlaceOrderMS</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>MyPlaceOrderMS</name>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
```



```
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<finalName> MyPlaceOrderMS </finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

MicroService Name	Rest Endpoints
MyPlaceOrderMS	http://localhost:7000/placeOrder
MyPlaceOrderMS	http://localhost:7000/myorders/{userId}